Deployment & Infrastructure Plan v2.0 — Budget-Optimized, Sprint-Ready

Document ID: DEPLOY-INFRA-v2.0
Version: 2.0
Date: 2026-03-04
Status: Draft
Author: Engineering Team
Related Documents: v1.0 | BRD | TDD

Table of Contents

1. Executive Summary
2. Constraints & Design Principles
3. Phase Feature Map
4. PRIMARY APPROACH: VPS + Managed Edge
5. Architecture Overview
6. Component Breakdown
7. Cloud Provider Comparison
8. Phase-wise Implementation Details
9. AI-Powered Answer Analysis
10. Communication: Email & WhatsApp
11. DevOps & Deployment
12. Monitoring & Observability
13. Cost Summary — Primary Approach
14. Scaling Playbook
15. ALTERNATE APPROACH: PaaS-Centric
16. Comparison: Primary vs Alternate
17. Pre-Deployment Checklist
18. Revision History

---

1. Executive Summary

This plan replaces the tiered approach in v1.0 with a **single, phase-proof infrastructure** designed around two realities:

- **Day-to-day:** 20 students + 3 admins (negligible load)
- **Campaign peaks:** Up to 500 concurrent users (rare, 3–5 days/month when it happens)

Each of the 5 phases is implemented over 2 sprints (4 weeks), giving a total timeline of ~20 weeks. The infrastructure is **deployed once on day 1 and remains stable** across all phases — no migrations, no environment changes.

Metric	Primary Approach	Alternate Approach

Metric	Primary Approach	Alternate Approach
Day-to-day cost	~$16–18/mo	~$25–55/mo
Campaign month cost	~$21–23/mo	~$35–65/mo
First-year cost	~$280–350	~$450–800
Deploy complexity	Medium (Docker + SSH)	Low (git push)
DevOps knowledge	Medium	Low

---

2. Constraints & Design Principles

Constraint	Implication
5 phases × 4 weeks each = 20 weeks	Infrastructure supports every phase from day 1 — no mid-project migrations
Budget-first	Minimize recurring costs; prefer free tiers and self-hosting over managed services
20 students + 3 admins daily	A single server handles this trivially
500 concurrent (rare, 3–5 days/month)	Temporarily scale up the VPS, then scale back down (hourly billing)
Production stability	Automated backups, health checks, zero-downtime deploys
Easy to deploy & maintain	Docker Compose for app services + native PostgreSQL on the VPS
SaaS-ready from day 1	Multi-tenant isolation already built; infrastructure supports custom domains and per-client branding

Core Principle: One VPS runs everything. Cloudflare handles the edge. Scale vertically for campaigns (hourly billing means you only pay for the hours at the higher tier). Move to horizontal scaling only when revenue justifies it.

---

3. Phase Feature Map

All phases share the same infrastructure, deployed on day 1. No environment changes between phases.

Phase	Timeline	Features	Infrastructure Required	New Services
Phase 1	Weeks 1–4	MCQ tests, courses, PDF materials, recorded videos	**API server** — handles all REST endpoints for CRUD and business logic	None (base stack)

Phase	Timeline	Features	Infrastructure Required	New Services
			PostgreSQL — stores all structured data (users, tests, questions, courses, submissions)	
			Redis — powers BullMQ job queues for async tasks (notifications, AI analysis)	
			Cloudflare R2 — S3-compatible object storage for PDFs, video files, and tenant logos (zero egress cost for student downloads)	
			Cloudflare Workers — global CDN hosting for both React SPAs (student-web, admin-web)	
			Caddy — reverse proxy with automatic HTTPS via Let's Encrypt	
Phase 2	Weeks 5–8	Live class meetings (up to 30 members, more for campaigns)	**100ms / Dyte SDK** — embeddable WebRTC video conferencing, handles all media routing and scaling server-side	**100ms or Dyte** — hosted WebRTC service with React SDK; free tier covers ~5 one-hour sessions/month with 30 participants
Phase 3	Weeks 9–12	Coding tests with real-time compiler and test case validation	**Docker Engine (on VPS)** — runs sandboxed code execution containers with network isolation, memory/CPU limits, and timeouts	**Judge Worker** — BullMQ-based worker process on the same VPS that dequeues submissions, executes code in Docker containers, compares output against test cases, and triggers AI analysis
			Pre-pulled language images — Python, Node.js, Java, C/C++, Go Docker images (~1.2 GB total) ready for instant container spin-up	
Phase 4	Weeks 13–16	Student mobile app (Android, optionally iOS)	**Capacitor** — wraps the existing React student-web in a native WebView shell; same codebase, same API, no new backend infrastructure	**Google Play Store account** ($25 one-time) — for publishing the Android APK/AAB

Phase	Timeline	Features	Infrastructure Required	New Services
			Firebase Cloud Messaging (free) — push notifications to mobile devices	**Apple Developer account** ($99/year, optional) — only if iOS distribution is needed
Phase 5	Weeks 17–20	SaaS white-label model with per-client branding	**Cloudflare for SaaS** — automatic TLS certificate provisioning for tenant custom domains (tenants point their CNAME, Cloudflare handles SSL)	**Cloudflare for SaaS add-on** ($2/mo for first 100 custom hostnames)
			Tenant branding pipeline — each client gets unique logo, colors, fonts, and optional custom CSS applied dynamically via the existing `brandingConfig` JSON on the `Tenant` model	**Razorpay / Stripe** — payment gateway for tenant subscription billing (2–3% transaction fee, no monthly cost)

---

4. PRIMARY APPROACH: VPS + Managed Edge

Philosophy

Run all application services (API, Redis, Judge worker) via Docker Compose on a single **Hetzner Cloud VPS**. Install **PostgreSQL natively** on the host (not in Docker — see Section 6.3 for rationale). Serve all frontend assets from **Cloudflare Workers (Static Assets)** — free, global CDN. Store files on **Cloudflare R2** — free, zero egress. Use **free tiers** for email, AI, monitoring, and live meetings.

---

5. Architecture Overview

```
┌──────────────────────────────────────────────────────────────────┐
│                    Cloudflare (Free Plan)                        │
│  DNS · CDN · DDoS Protection · SSL · Workers Static Assets       │
│                                                                  │
│  ┌────────────────┐  ┌────────────────┐                         │
│  │ student.       │  │ admin.         │  ← Cloudflare Workers   │
│  │ yourdomain.com │  │ yourdomain.com │    (Static Assets, Free)│
│  └────────┬───────┘  └────────┬───────┘                         │
│           └──────────┬────────┘                                  │
│                      ▼                                           │
│           ┌────────────────┐                                     │
│           │ api.           │  ← Cloudflare DNS proxied           │
│           │ yourdomain.com │                                     │
│           └────────┬───────┘                                     │
└────────────────────┼─────────────────────────────────────────────┘
                     │
                     ▼
┌──────────────────────────────────────────────────────────────────┐
│              Hetzner VPS — CX32 (~€14/mo after Apr 2026)         │
```

```
│              4 vCPU · 8 GB RAM · 80 GB NVMe SSD                  │
│                                                                  │
│  ┌──────────────────────────────────────────────────────────┐   │
│  │           Native (installed via apt)                     │   │
│  │                                                          │   │
│  │  ┌──────────────────────────────────────────────────┐    │   │
│  │  │  PostgreSQL 16                                   │    │   │
│  │  │  Data: /var/lib/postgresql/16/main               │    │   │
│  │  │  Backup: daily pg_dump → Cloudflare R2           │    │   │
│  │  └──────────────────────────────────────────────────┘    │   │
│  └──────────────────────────────────────────────────────────┘   │
│                                                                  │
│  ┌──────────────────────────────────────────────────────────┐   │
│  │           Docker Compose                                 │   │
│  │                                                          │   │
│  │  ┌─────────┐   ┌──────────┐   ┌─────────┐              │   │
│  │  │ Caddy   │   │ Fastify  │   │ Redis   │              │   │
│  │  │ Reverse │→  │ API      │→  │ 7.x     │              │   │
│  │  │ Proxy   │   │ (Node 20)│   │ (queues)│              │   │
│  │  │ + SSL   │   │          │   │         │              │   │
│  │  └─────────┘   └──────┬───┘   └─────────┘              │   │
│  │                       │                                  │   │
│  │                       ▼                                  │   │
│  │                 ┌──────────────┐                         │   │
│  │                 │ Judge Worker │  ← Docker-in-Docker     │   │
│  │                 │ (BullMQ)     │    sandboxed execution  │   │
│  │                 └──────────────┘                         │   │
│  └──────────────────────────────────────────────────────────┘   │
│                                                                  │
│  Cron: pg_dump → gzip → rclone → Cloudflare R2 (daily 2 AM)     │
└──────────────────────────────────────────────────────────────────┘


External Services (All Free Tiers):
  ├── Cloudflare R2        → File storage (PDFs, videos, logos, backups)
  ├── Resend               → Email invites & notifications (100/day free)
  ├── Google Gemini API    → AI answer pattern analysis
  ├── 100ms / Dyte         → Live class meetings (10K participant-min/mo free)
  ├── Sentry               → Error tracking
  ├── BetterStack          → Uptime monitoring
  └── Firebase Cloud Msg.  → Mobile push notifications (free)
```

---

6. Component Breakdown

6.1 VPS — Hetzner CX32

Spec	Value
vCPU	4 (shared Intel)
RAM	8 GB
Storage	80 GB NVMe SSD

Spec	Value
Traffic	20 TB/month included
OS	Ubuntu 24.04 LTS
Cost (current, before Apr 2026)	€6.80/mo (~$7.50)
Cost (after Apr 1, 2026)	€13.99/mo (~$15.40)
Hourly rate (after Apr 2026)	€0.0224/hr
Location	Falkenstein (EU) or Ashburn (US East) — choose closest to your users

> **Note:** Hetzner announced a price adjustment effective April 1, 2026. All cost estimates in this document use the **post-April pricing** to be accurate for the project timeline.

RAM allocation (estimated):

Process	RAM Usage
Ubuntu OS + overhead	~500 MB
PostgreSQL 16 (native)	~500 MB
Docker Engine	~300 MB
Fastify API (Node 20)	~200 MB
Redis 7	~100 MB
Judge Worker (Node 20)	~200 MB
Caddy reverse proxy	~50 MB
Reserved for code execution containers	**~6 GB**

6.2 Reverse Proxy — Caddy (Docker)

Caddy over Nginx because:

- **Automatic HTTPS** via Let's Encrypt — zero certificate configuration
- **Automatic renewal** — no cron jobs or certbot
- **Simple config** — 5 lines vs 50+ lines of Nginx

```
api.yourdomain.com {
    reverse_proxy api:3000
}
```

6.3 Database — PostgreSQL 16 (Native on VPS)

Why NOT in Docker?

The standard production practice is to run databases natively on the host, not inside Docker containers. The reasons:

Concern	Docker DB	Native DB
I/O performance	Storage driver adds abstraction layer; benchmarks show 5–15% overhead on write-heavy workloads	Direct disk access, no overhead
Data safety	Accidental `docker compose down -v` deletes all data	Data persists in `/var/lib/postgresql/` regardless of Docker state
Resource contention	Competes with app containers for cgroup limits	OS manages resources independently
Scaling path	Must migrate out of Docker to scale	Already native; migrate to managed service when needed
Upgrades	Requires careful volume management across image versions	`apt upgrade` with standard PostgreSQL tooling
Backups	Must exec into container or mount volumes	Standard `pg_dump`, WAL archiving, `pg_basebackup`
Monitoring	Extra hops for metrics	Direct access to `pg_stat_*` views

Setup on VPS:

```
sudo apt install postgresql-16 postgresql-client-16
sudo -u postgres createuser --pwprompt enrich
sudo -u postgres createdb --owner=enrich enrich_skills
```

Backup strategy (cron, daily at 2 AM):

```
#!/bin/bash
TIMESTAMP=$(date +%Y%m%d_%H%M%S)
pg_dump -U enrich enrich_skills | gzip > /tmp/backup_$TIMESTAMP.sql.gz
rclone copy /tmp/backup_$TIMESTAMP.sql.gz r2:enrich-backups/postgres/
rm /tmp/backup_$TIMESTAMP.sql.gz
rclone delete --min-age 7d r2:enrich-backups/postgres/
```

6.4 Redis 7 (Docker)

- **Purpose:** BullMQ job queues (judge submissions, notification dispatch, AI analysis triggers)
- **Memory:** ~50–100 MB for 20 users
- **Persistence:** RDB snapshots every 5 minutes (default)
- Redis is fine in Docker — it's ephemeral queue data, not critical persistent state. If the container restarts, pending jobs are re-enqueued.

6.5 Frontend Hosting — Cloudflare Workers (Static Assets)

Cloudflare is deprecating the standalone "Pages" product in favor of **Workers with Static Assets**. The architecture is the same — your React build output is deployed to Cloudflare's global network — but the deployment method changes slightly.

Factor	Old (Cloudflare Pages)	New (Workers Static Assets)
Deployment	Git-based or `wrangler pages deploy`	`wrangler deploy` with `wrangler.toml` config
Static asset cost	Free	Free (static asset requests remain free)
CDN	Global	Global (same infrastructure)
Custom domains	Dashboard config	`wrangler.toml` routes config
Server-side logic	Pages Functions	Full Workers capabilities (Durable Objects, Cron, etc.)
Free tier	100K requests/day	100K requests/day (same)

Migration is minimal. Create a `wrangler.toml` in each frontend app:

```
name = "enrich-student"
compatibility_date = "2026-03-04"

[assets]
directory = "./dist"
```

Deploy with: `wrangler deploy`

The architecture is future-proof. Whether it's called Pages or Workers, the result is identical: static React bundles served from 300+ Cloudflare edge locations with automatic SSL, zero bandwidth cost, and sub-50ms load times globally.

6.6 File Storage — Cloudflare R2

Factor	Detail
Free tier	10 GB storage, 1M Class A ops, 10M Class B ops/month
Egress	$0 (zero egress — critical for video/PDF downloads)
Paid	$0.015/GB/month beyond 10 GB
Compatibility	S3-compatible API — use `@aws-sdk/client-s3`

Storage layout:

Prefix	Content	Access Pattern
`materials/`	Course PDFs, documents	Authenticated download via presigned URL
`videos/`	Recorded batch videos (MP4)	Presigned URL streaming
`logos/{tenantId}/`	Per-tenant branding assets (logo, favicon)	Public read
`backups/`	Daily PostgreSQL dumps	Private, 7-day retention
`reports/`	Generated AI/PDF reports	Authenticated download

6.7 DNS, CDN, SSL — Cloudflare (Free)

- DNS management for all subdomains
- DDoS protection (always-on)
- SSL termination at edge
- HTTP/2 and HTTP/3
- For Phase 5 SaaS: **Cloudflare for SaaS** add-on ($2/mo for first 100 custom hostnames, $0.10/hostname after)

---

7. Cloud Provider Comparison

7.1 VPS Comparison — 8 GB RAM Tier

The 8 GB tier is the target for running API + PostgreSQL + Redis + Judge on a single server.

Provider	Plan	vCPU	RAM	Storage	Transfer	Monthly	Hourly	Region Nearest to India
Hetzner	CX32	4	8 GB	80 GB NVMe	20 TB	**€13.99 (~$15.40)**	€0.0224	Ashburn (US) / Falkenstein (EU)
DigitalOcean	Basic 8 GB	4	8 GB	160 GB SSD	5 TB	**$48.00**	$0.0714	Bangalore (India)
AWS Lightsail	8 GB	2	8 GB	160 GB SSD	5 TB	**$44.00**	~$0.0603	Mumbai (India)
AWS EC2	t3.large	2	8 GB	EBS (extra $)	Egress-based	**~$60.00+**	$0.0832	Mumbai
GCP	e2-standard-2	2	8 GB	10 GB (extra $)	Egress-based	**$48.92**	$0.0670	Mumbai

Provider	Plan	vCPU	RAM	Storage	Transfer	Monthly	Hourly	Region Nearest to India
GCP (1yr CUD)	e2-standard-2	2	8 GB	10 GB (extra $)	Egress-based	**$30.82**	N/A (committed)	Mumbai

7.2 VPS Comparison — 4 GB RAM Tier (Minimum Viable)

Provider	Plan	vCPU	RAM	Storage	Transfer	Monthly
Hetzner	CX22	2	4 GB	40 GB NVMe	20 TB	**~€7 (~$7.70)**
DigitalOcean	Basic 4 GB	2	4 GB	80 GB SSD	4 TB	**$24.00**
AWS Lightsail	4 GB	2	4 GB	80 GB SSD	4 TB	**$24.00**
GCP	e2-medium	1	4 GB	10 GB (extra $)	Egress-based	**$24.46**

7.3 Detailed Provider Comparison

Factor	Hetzner	DigitalOcean	AWS (Lightsail/EC2)	GCP
8 GB monthly cost	**€13.99 ($15.40)**	$48	$44–60	$49–31 (CUD)
Price-to-perf ratio	**Best**	Good	Moderate	Moderate
Included bandwidth	**20 TB**	5 TB	5 TB	Egress-based ($0.12/GB)
Hourly billing	Yes (capped)	Yes (per-second)	Yes (Lightsail capped)	Yes (per-second)
Campaign scale-up cost	**~€5 for 5 days**	~$15 for 5 days	~$13 for 5 days	~$15 for 5 days
Managed PostgreSQL	No	Yes ($15+/mo)	Yes (RDS $15+/mo)	Yes (Cloud SQL $10+/mo)
Docker support	Yes (manual install)	Yes (1-click image)	Yes (manual)	Yes (manual)
India data center	No (nearest: Singapore +40%)	**Yes (Bangalore)**	**Yes (Mumbai)**	**Yes (Mumbai)**
EU data center	**Yes (Germany, Finland)**	Yes (Amsterdam, Frankfurt)	Yes (Frankfurt, Ireland)	Yes (Belgium, Finland)
US data center	Yes (Ashburn)	Yes (multiple)	Yes (multiple)	Yes (multiple)
Control panel UX	Good	**Best**	Complex	Complex

Factor	Hetzner	DigitalOcean	AWS (Lightsail/EC2)	GCP
Documentation	Good	**Best**	Good (verbose)	Good (verbose)
Firewall	Free (cloud firewall)	Free	Free (security groups)	Free (VPC firewall)
Snapshots	€0.012/GB/mo	$0.06/GB/mo	Included (5 snapshots)	$0.026/GB/mo
DDoS protection	Basic (free)	Basic (free)	AWS Shield (free basic)	Cloud Armor (paid)
Free tier	No	No	12-month (t2.micro, etc.)	Always-free (e2-micro)
Vendor lock-in	**None**	Low	High	High
Hidden costs	**None**	None	Egress, EBS, IPs	Egress, disks, IPs

7.4 Recommendation

Situation	Best Provider	Why
Cheapest possible (your case)	**Hetzner**	3x cheaper than alternatives for equivalent specs
Users primarily in India, latency-sensitive	**DigitalOcean (Bangalore)**	Lowest latency; 3x the cost of Hetzner
Need AWS ecosystem (future enterprise clients)	**AWS Lightsail**	Easy on-ramp to full AWS; moderate premium
Already using GCP / need GCP-specific services	**GCP**	Only if you need Cloud Run, BigQuery, etc.

For this project: Hetzner CX32 — the 3x cost saving outweighs the latency difference. Your students access the API through Cloudflare's CDN anyway, which caches and accelerates requests globally. Database queries add ~80–120ms latency from India to EU (Falkenstein), which is acceptable for an LMS. If latency becomes an issue, move to Hetzner Singapore (+40% cost, still cheaper than alternatives).

---

8. Phase-wise Implementation Details

8.1 Phase 1: MCQ, Courses, PDF Materials, Recorded Videos (Weeks 1–4)

Already built. The current codebase handles:

- Question bank (MCQ + coding questions) with CRUD
- Test creation with variants and allocations
- Course management with chapters, topics, and materials
- Batch management with members and test assignments
- File upload/download for materials

- Student portal with test taking, course viewing
- Admin dashboard with full management

Infrastructure needed: Base stack only (API + PostgreSQL + Redis + R2 + Caddy + Cloudflare Workers).

Deployment steps:

1. Provision Hetzner CX32 VPS (Ubuntu 24.04)
2. Install PostgreSQL 16 natively (`apt install postgresql-16`)
3. Install Docker Engine + Docker Compose
4. Deploy `docker-compose.prod.yml` (API, Redis, Caddy, Judge worker)
5. Deploy student-web and admin-web to Cloudflare Workers (Static Assets)
6. Configure R2 bucket and migrate `storage.ts` to S3 client
7. Set up daily PostgreSQL backup cron to R2
8. Configure Cloudflare DNS records
9. Set up Sentry + BetterStack monitoring

Video handling:

- Recorded videos uploaded to R2 via presigned upload URLs (multipart for >100 MB)
- Students stream via presigned download URLs (time-limited, 1-hour expiry)
- R2's zero egress cost means no per-download charges regardless of student count

8.2 Phase 2: Live Class Meetings (Weeks 5–8)

Recommended: 100ms (hosted WebRTC service)

Factor	Detail
Free tier	10,000 participant-minutes/month
Paid rate	$0.004/min (audio), $0.008/min (video)
SDK	React SDK (`@100mslive/react-sdk`) — embed in student-web and admin-web
Features	Video/audio, screen sharing, chat, recording, virtual backgrounds
Max capacity	100 peers (mesh), 10,000+ viewers (SFU mode for campaigns)

Cost math for 30-member classes:

Scenario	Participant-Minutes	Free Tier Covers?	Overage Cost
1 session/week (1 hr)	7,200/mo	Yes (10K free)	$0
2 sessions/week (1 hr)	14,400/mo	Partial	~$35/mo
Campaign: 500 people × 1 hr	30,000 per session	No	~$240/session

Campaign alternative for large audiences: Stream via **YouTube Live** (free, unlimited viewers) and embed the player in the student-web. Reserve 100ms for interactive classes with ≤30 members.

Integration (fits in 4-week sprint):

1. Admin creates a "Live Class" schedule event in the batch calendar
2. Backend generates a 100ms room via REST API, stores room ID
3. Student-web embeds the `<HMSRoomProvider>` React component
4. Admin-web has a "Start Class" button that creates/joins the room
5. Optional: recordings saved to R2 via 100ms webhook

Alternative services:

Service	Free Tier	Paid Rate	India Presence	Notes
100ms	10K min/mo	$0.004–0.008/min	Yes	Best React SDK
Dyte	10K min/mo	$0.004/min	Yes (Indian company)	Good API, competitive
LiveKit Cloud	50 GB bandwidth/mo	Usage-based	No (self-host option)	Open source, self-hostable
Jitsi (JaaS by 8x8)	5K min/mo, 25 users/room	Usage-based	No	Mature, open source core

8.3 Phase 3: Coding Tests with Real-time Compiler (Weeks 9–12)

Architecture: Judge worker runs as a Docker container on the same VPS, picks jobs from BullMQ (Redis), executes student code in isolated Docker containers.

```
Student writes code in Monaco Editor (student-web)
        │
        ▼
  POST /api/v1/attempts/:id/submit-code
        │
        ▼
  API creates Submission (status: 'pending')
  API pushes job to BullMQ "judge" queue
        │
        ▼
  Judge Worker picks job from Redis queue
        │
        ├── Pull question's test cases from PostgreSQL
        ├── For each test case:
        │     ├── docker run --rm --network=none --memory=256m --cpus=0.5
        │     │     --read-only --security-opt=no-new-privileges
        │     │     {lang-image} < stdin(input) > stdout(output)
        │     ├── Compare stdout to expected output
        │     ├── Record: passed, executionTimeMs, memoryUsageMb, actualOutput
        │     └── Kill container after timeLimit (default 5s)
        ├── Calculate weighted score
        ├── Update Submission in PostgreSQL (status: 'completed', score)
        │
        ▼
  Push AI analysis job to "ai-analysis" queue
```

```
        │
        ▼
AI Worker calls Gemini API with code + test results
Stores structured analysis in Submission.aiRatings JSON field
        │
        ▼
Notify student (WebSocket/SSE or email if configured)
```

Pre-pulled Docker images (language runtimes):

Language	Image	Size
Python 3.12	`python:3.12-slim`	~150 MB
Node.js 20	`node:20-slim`	~200 MB
Java 21	`eclipse-temurin:21-jre`	~250 MB
C/C++ (GCC 13)	`gcc:13-bookworm`	~400 MB
Go 1.22	`golang:1.22-alpine`	~250 MB
Total		**~1.25 GB**

Security hardening per execution container:

Measure	Flag	Purpose
No network	`--network=none`	Prevent internet access, API calls, data exfiltration
Memory cap	`--memory=256m`	Prevent OOM from infinite allocation
CPU cap	`--cpus=0.5`	Prevent CPU starvation of other services
Read-only FS	`--read-only`	Prevent writing to filesystem (except /tmp via tmpfs)
No privilege escalation	`--security-opt=no-new-privileges`	Prevent sudo/setuid abuse
Non-root user	`--user=1000:1000`	Run as unprivileged user
Timeout	Kill after `timeLimit` seconds	Default 5s, configurable per question

Concurrency: For 20 students, the judge worker runs with BullMQ concurrency of 1. Even if all 20 submit simultaneously, at ~3s per submission the entire queue clears in under a minute.

8.4 Phase 4: Student Mobile App (Weeks 13–16)

Strategy: Capacitor wrapping the existing React student-web.

Factor	Detail
Code reuse	100% — same React codebase, same CSS, same API calls

Factor	Detail
New infrastructure	None — mobile app connects to the same API endpoint
Build time	2–3 days to set up Capacitor + publish
Native features	Push notifications (Firebase), camera (future proctoring), biometrics

Steps:

1. `pnpm --filter student-web add @capacitor/core @capacitor/cli`
2. `npx cap init "Enrich Skills" com.enrichskills.app`
3. `npx cap add android` (and `ios` if needed)
4. Add Firebase Cloud Messaging plugin for push notifications
5. Build: `pnpm --filter student-web build && npx cap sync && npx cap open android`
6. Publish to Google Play

App Store costs:

Store	Fee	Review	Notes
Google Play	**$25 one-time**	Hours–1 day	Start here
Apple App Store	$99/year	1–3 days	Add when iOS demand exists

8.5 Phase 5: SaaS / White-Label with Per-Client Branding (Weeks 17–20)

Current multi-tenant foundation (already built):

- `Tenant` model with `slug`, `domain`, `brandingConfig`, `featureFlags`, `status`
- `tenantId` on all core models (User, Test, Question, Course, Batch, etc.)
- `requireTenant()` middleware on all API routes
- JWT includes `tenantId` and `role`

Per-client branding — what changes for each client:

The existing `Tenant.brandingConfig` JSON field supports per-client customization:

Branding Element	Stored In	How Applied	Admin UI Needed?
Logo	`brandingConfig.logoUrl` → file in R2 `logos/{tenantId}/`	`<img>` in header/login	Yes — upload in tenant settings
Favicon	`brandingConfig.faviconUrl` → file in R2	`<link rel="icon">` injected dynamically	Yes — upload in tenant settings
Primary color	`brandingConfig.primaryColor` (hex)	CSS custom property `--primary-color`	Yes — color picker
Secondary color	`brandingConfig.secondaryColor` (hex)	CSS custom property `--secondary-color`	Yes — color picker

Branding Element	Stored In	How Applied	Admin UI Needed?
Font family	`brandingConfig.fontFamily`	CSS custom property `--font-family` + Google Fonts import	Yes — dropdown
Custom CSS	`brandingConfig.customCss` (sanitized)	Injected via `<style>` tag	Yes — code editor (advanced)
App name	`Tenant.name`	Shown in header, emails, page title	Yes — text input
Custom domain	`Tenant.domain`	Cloudflare for SaaS (CNAME + auto TLS)	Yes — domain input + CNAME instructions
Email branding	Template variables from tenant config	Logo + colors in React Email templates	Automatic from above settings

How branding works at runtime:

1. User visits `learn.clientbrand.com` (or `client.yourdomain.com`)
2. Frontend loads → calls `GET /api/v1/tenants/resolve?domain={hostname}`
3. API looks up `Tenant` by `domain` field → returns branding config
4. Frontend applies CSS custom properties:

```
:root {
  --primary-color: {tenant.primaryColor};
  --secondary-color: {tenant.secondaryColor};
  --font-family: {tenant.fontFamily};
}
```

5. Logo, favicon, and app name rendered from config
6. Custom CSS injected (sanitized to prevent XSS)

Custom domain setup (Cloudflare for SaaS):

Step	Who	What
1	You	Enable Cloudflare for SaaS on your zone ($2/mo)
2	You	Add a fallback origin (`custom-saas.yourdomain.com → VPS IP`)
3	Client	Adds CNAME: `learn.theirclient.com → custom-saas.yourdomain.com`
4	Cloudflare	Auto-provisions TLS certificate for `learn.theirclient.com`
5	API	Resolves the hostname to the correct tenant, serves branded content

SaaS billing integration:

- Razorpay (India) or Stripe (global) for subscription payments
- Webhook-based: payment events → update `Tenant.status` and `Tenant.plan`
- No monthly platform fee — only 2–3% transaction fee per payment

---

9. AI-Powered Answer Analysis

9.1 Model Selection

Model	Cost per 1M Tokens	Best For
Gemini 2.0 Flash	$0.10 input / $0.40 output	All analysis — cheapest capable model
GPT-4o mini	$0.15 input / $0.60 output	Fallback if Gemini is unavailable

Recommendation: Use **Gemini 2.0 Flash** exclusively. For 20 students, the monthly AI cost is under $1.

9.2 MCQ Answer Pattern Analysis

After a student completes an MCQ test, the AI analyzes:

Analysis Dimension	Input Data	Output
Topic-wise proficiency	Questions grouped by topic/tag + student answers	Score per topic (e.g., "Arrays: 8/10, Recursion: 3/10")
Misconception detection	Wrong answers + correct answers + question text	Common misconceptions identified with explanations
Time analysis	Time spent per question (if tracked)	Patterns like "rushed hard questions, slow on easy ones"
Cohort comparison	Student answers vs batch average	Percentile ranking, areas below/above batch average
Improvement trend	Historical attempts across multiple tests	"Improving in X, declining in Y" over time
Strengths & weaknesses	All of the above	Top 3 strengths, top 3 areas to improve, actionable advice

9.3 Prompt Strategy

Per-test analysis (~800 tokens input → ~500 tokens output):

```
You are an educational assessment analyst. Analyze this student's MCQ test
performance and identify patterns in their answers.

Test: {testName}
Subject: {subject}
Total Questions: {total} | Correct: {correct} | Score: {percentage}%
```

```
Questions and Answers:
{for each question:}
- Q{n} [Topic: {topic}] [Difficulty: {easy|medium|hard}]:
  "{questionText}"
  Options: A) ... B) ... C) ... D) ...
  Student chose: {studentAnswer} | Correct: {correctAnswer} | Result: ✓/✗
  Time spent: {seconds}s
{end}

Respond with this exact JSON structure:
{
  "topicScores": { "{topic}": { "correct": n, "total": n, "percentage": n } },
  "strengths": ["specific strength with evidence from answers"],
  "weaknesses": ["specific weakness with evidence from answers"],
  "misconceptions": ["identified misconception based on wrong answer patterns"],
  "answerPatterns": ["observed pattern, e.g. 'consistently picks longest option'"],
  "recommendations": ["actionable study recommendation"],
  "overallAssessment": "2-3 sentence personalized summary"
}
```

9.4 Code Submission Analysis (Phase 3)

For coding questions, the AI additionally evaluates:

Dimension	Rating	What It Measures
Code quality	1–10	Readability, naming conventions, structure, DRY principles
Problem solving	1–10	Algorithm choice, approach, edge case handling
Efficiency	1–10	Time complexity, space complexity, optimal vs brute force
Correctness	1–10	Test case pass rate, partial correctness, error handling

Stored in `Submission.aiRatings` (JSON field, already in the Prisma schema).

9.5 Cost Estimate

Scale	Tests/Month	AI Cost/Month
20 students, 2 tests/month	40 analyses	**$0.03**
20 students, 10 tests/month	200 analyses	**$0.15**
500 students (campaign), 1 test	500 analyses	**$0.40**
Phase 5: 10 tenants × 50 students × 5 tests	2,500 analyses	**$2.00**

AI cost is negligible at this scale — well under $5/month for all foreseeable usage.

9.6 Report Types

Report	Trigger	Content	Storage

Report	Trigger	Content	Storage
Per-test report	Student completes test	Topic scores, strengths, weaknesses, misconceptions	`Submission.aiRatings` JSON
Student progress report	Admin requests or monthly cron	Trend across all tests, skill map, improvement areas	Generated on-demand, cached in R2
Batch report	Admin views batch analytics	Cohort averages, top performers, common weak topics	Generated on-demand

10. Communication: Email & WhatsApp

10.1 Email — Resend

Factor	Detail
Service	Resend
Free tier	100 emails/day, 3,000/month
Paid	$20/month for 50,000 emails/month
Integration	REST API or SMTP relay (drop-in for existing `nodemailer` in the codebase)
Templates	React Email templates (JSX-based, type-safe)

For 20 students + 3 admins, the free tier (100 emails/day) is more than sufficient.

Email events:

Event	Trigger	Template
Platform invite	Admin sends invite	Welcome email with signup link
Test assigned	Admin allocates test to batch/student	Test name, deadline, link to attempt
Test results ready	AI analysis complete	Score summary, link to detailed report
Course material published	Admin publishes new content	Course name, material title, link
Live class scheduled	Admin creates schedule event	Date, time, join link
Password reset	User requests	Time-limited reset link

10.2 WhatsApp — Manual via WhatsApp Web

Per your stated preference, WhatsApp broadcasting is done **manually** through WhatsApp Web to your group:

- **No WhatsApp API integration needed** — saves $12+/month
- **No infrastructure or code changes**
- Admin copies key information from the dashboard and broadcasts to the WhatsApp group

- This works well for 20 students in a single batch

Future automation path (when scaling to SaaS / multiple clients):

Scale	Provider	Monthly Cost	Integration Effort
1–5 tenants	Interakt (Indian BSP)	₹999 (~$12)	Simple REST API, 2–3 days
5–50 tenants	Wati	$40	Better API, webhooks, 3–5 days
50+ tenants	Meta Cloud API (direct)	Per-conversation only	More setup, cheapest at scale

---

11. DevOps & Deployment

11.1 Docker Compose — Production Stack

PostgreSQL runs **natively on the host**. Docker Compose manages only the application services:

```
# docker-compose.prod.yml
services:
  caddy:
    image: caddy:2-alpine
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./Caddyfile:/etc/caddy/Caddyfile
      - caddy_data:/data
    depends_on:
      - api
    restart: unless-stopped

  api:
    build:
      context: .
      dockerfile: apps/api/Dockerfile
    environment:
      -
DATABASE_URL=postgresql://enrich:${DB_PASSWORD}@host.docker.internal:5432/enrich_sk
ills
      - REDIS_URL=redis://redis:6379
      - JWT_SECRET=${JWT_SECRET}
      - R2_ACCESS_KEY=${R2_ACCESS_KEY}
      - R2_SECRET_KEY=${R2_SECRET_KEY}
      - R2_BUCKET=${R2_BUCKET}
      - R2_ENDPOINT=${R2_ENDPOINT}
      - RESEND_API_KEY=${RESEND_API_KEY}
      - GEMINI_API_KEY=${GEMINI_API_KEY}
      - MEETING_API_KEY=${MEETING_API_KEY}
    extra_hosts:
      - "host.docker.internal:host-gateway"
    depends_on:
      - redis
```

```
    restart: unless-stopped

  judge-worker:
    build:
      context: .
      dockerfile: services/judge/Dockerfile
    environment:
      -
DATABASE_URL=postgresql://enrich:${DB_PASSWORD}@host.docker.internal:5432/enrich_sk
ills
      - REDIS_URL=redis://redis:6379
    extra_hosts:
      - "host.docker.internal:host-gateway"
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
    depends_on:
      - redis
    restart: unless-stopped

  redis:
    image: redis:7-alpine
    command: redis-server --maxmemory 256mb --maxmemory-policy allkeys-lru
    volumes:
      - redis_data:/data
    restart: unless-stopped

volumes:
  caddy_data:
  redis_data:
```

> **Note:** `host.docker.internal:host-gateway` allows Docker containers to connect to the host's native PostgreSQL instance. On Linux, this maps to the host's IP. PostgreSQL's `pg_hba.conf` must allow connections from the Docker bridge network (typically `172.17.0.0/16`).

11.2 Deployment Workflow

Option A: Simple SSH deploy script (recommended for small team)

```
#!/bin/bash
# deploy.sh — run from local machine
ssh root@your-vps-ip << 'DEPLOY'
  cd /opt/enrich-skills
  git pull origin main
  docker compose -f docker-compose.prod.yml build --no-cache api judge-worker
  docker compose -f docker-compose.prod.yml up -d
  # Run Prisma migrations against the native PostgreSQL
  docker compose -f docker-compose.prod.yml exec -T api npx prisma migrate deploy
  docker system prune -f
DEPLOY
```

Option B: GitHub Actions CI/CD

```yaml
name: Deploy
on:
  push:
    branches: [main]

jobs:
  deploy-api:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Deploy to VPS
        uses: appleboy/ssh-action@v1
        with:
          host: ${{ secrets.VPS_HOST }}
          username: root
          key: ${{ secrets.VPS_SSH_KEY }}
          script: |
            cd /opt/enrich-skills
            git pull origin main
            docker compose -f docker-compose.prod.yml build --no-cache api judge-
worker
            docker compose -f docker-compose.prod.yml up -d
            docker compose -f docker-compose.prod.yml exec -T api npx prisma
migrate deploy

  deploy-frontends:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        app: [student-web, admin-web]
        include:
          - app: student-web
            project: enrich-student
          - app: admin-web
            project: enrich-admin
    steps:
      - uses: actions/checkout@v4
      - uses: pnpm/action-setup@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: pnpm
      - run: pnpm install --frozen-lockfile
      - run: pnpm --filter shared build
      - run: pnpm --filter ${{ matrix.app }} build
      - name: Deploy to Cloudflare Workers
        run: npx wrangler deploy
        working-directory: apps/${{ matrix.app }}
        env:
          CLOUDFLARE_API_TOKEN: ${{ secrets.CF_API_TOKEN }}
```

11.3 Environment Management

Environment	Database	API	Frontends	Cost
Local dev	Local PostgreSQL or Docker Postgres	`pnpm dev:api`	`pnpm dev:student` / `pnpm dev:admin`	$0
Production	Native PostgreSQL on VPS	Docker Compose on VPS	Cloudflare Workers (Static Assets)	~$16/mo

No staging environment to save cost. Use Neon database branching (free) if you need to test migrations before applying to production.

11.4 Required Dockerfiles

API (`apps/api/Dockerfile`):

```
FROM node:20-slim AS base
RUN corepack enable && corepack prepare pnpm@10.29.3 --activate
WORKDIR /app

FROM base AS deps
COPY pnpm-lock.yaml pnpm-workspace.yaml package.json ./
COPY packages/shared/package.json packages/shared/
COPY apps/api/package.json apps/api/
RUN pnpm install --frozen-lockfile --prod=false

FROM deps AS build
COPY packages/shared/ packages/shared/
COPY apps/api/ apps/api/
RUN pnpm --filter shared build
RUN pnpm --filter api build
RUN pnpm --filter api exec prisma generate

FROM node:20-slim AS runtime
WORKDIR /app
COPY --from=build /app/apps/api/dist ./dist
COPY --from=build /app/apps/api/prisma ./prisma
COPY --from=build /app/apps/api/node_modules ./node_modules
COPY --from=build /app/node_modules/.pnpm node_modules/.pnpm
EXPOSE 3000
CMD ["node", "dist/index.js"]
```

Judge Worker (`services/judge/Dockerfile`):

```
FROM node:20-slim
RUN apt-get update && apt-get install -y docker.io && rm -rf /var/lib/apt/lists/*
WORKDIR /app
COPY services/judge/package.json ./
RUN npm install --production
COPY services/judge/dist ./dist
CMD ["node", "dist/index.js"]
```

---

12. Monitoring & Observability

12.1 Monitoring Stack (All Free Tiers)

Layer	Tool	Free Tier Limits	Purpose
Error tracking	Sentry	5K events/mo	API errors, frontend crashes, unhandled rejections
Uptime	BetterStack	5 monitors, 3-min checks	Alert if API goes down
Logging	Docker logs + `journalctl`	Unlimited (local)	Debug issues on VPS
VPS metrics	Hetzner Cloud Console	Included	CPU, RAM, disk, network graphs
Performance	Fastify built-in logger	Unlimited	Request timing, slow query detection

12.2 Health Checks

```
GET /health    → { "status": "ok", "timestamp": "..." }           (liveness)
GET /ready     → { "status": "ready", "db": "ok", "redis": "ok" } (readiness)
```

BetterStack pings `/health` every 3 minutes. Email alert on 2 consecutive failures.

12.3 Alerting

Alert	Condition	Channel
API down	`/health` non-200 for 2 checks	Email (BetterStack)
High CPU	>80% for 5 minutes	Email (Hetzner alert)
Disk >70%	SSD usage threshold	Email (Hetzner alert)
Backup failure	Cron exit code ≠ 0	Email (cron MAILTO)
Judge queue backlog	BullMQ queue >50 items	Sentry alert

---

13. Cost Summary — Primary Approach

13.1 Day-to-Day Operation (20 Students + 3 Admins)

Uses post-April 2026 Hetzner pricing.

Component	Service	Monthly Cost
VPS (API + Redis + Judge)	Hetzner CX32 (4 vCPU, 8 GB)	€13.99 (~$15.40)

Component	Service	Monthly Cost
PostgreSQL	Native on VPS (included in VPS cost)	$0
Student Web	Cloudflare Workers (Static Assets)	$0
Admin Web	Cloudflare Workers (Static Assets)	$0
File storage	Cloudflare R2 (10 GB free)	$0
AI answer analysis	Gemini 2.0 Flash (~200 analyses/mo)	$0.15
Email	Resend (100/day free)	$0
WhatsApp	Manual (WhatsApp Web)	$0
Live meetings	100ms (10K min/mo free)	$0
DNS / CDN / SSL	Cloudflare Free	$0
Error tracking	Sentry Free	$0
Uptime monitoring	BetterStack Free	$0
Domain name	`.com` registration	~$1.00
TOTAL (day-to-day)		**~$16–17/mo**

13.2 Campaign Month (500 Concurrent, 3–5 Days)

Hetzner bills hourly. You resize the VPS for the campaign days and resize back after.

Campaign scaling math (5-day campaign in a 30-day month):

Period	VPS Tier	Hours	Hourly Rate	Cost
25 normal days	CX32 (4 vCPU, 8 GB)	600	€0.0224	€13.44
5 campaign days	CX42 (8 vCPU, 16 GB)	120	€0.0408	€4.90
Total VPS cost for month				**€18.34 (~$20.20)**
vs. full month at CX32				€13.99 ($15.40)
Campaign premium				**+€4.35 (~$4.80)**

Yes, you save significant money by scaling down. The campaign adds only ~$4.80 to the monthly VPS bill. Compare to running CX42 all month (€25.49 = $28.00).

Full campaign month breakdown:

Component	Normal Cost	Campaign Addition	Campaign Month Total
VPS (scaled up 5 days)	$15.40	+$4.80	$20.20
100ms (500 users × 3 sessions × 1 hr)	$0	~$720 (or $0 via YouTube Live)	$0–720

Component	Normal Cost	Campaign Addition	Campaign Month Total
AI analysis (500 extra students)	$0.15	+$0.40	$0.55
Email (campaign invites)	$0	$0 (within 100/day) or $20 (Resend Pro)	$0–20
Campaign month total			**$21–$741**

> **Recommendation:** For campaign live sessions with 500 people, use **YouTube Live embed** (free) instead of 100ms ($240/session). This brings the campaign month total to **~$21–23/mo**.

13.3 Cost by Phase (Incremental)

Phase	Weeks	New Infrastructure Cost	Running Total
Phase 1: MCQ, courses, PDFs, videos	1–4	$16/mo (base stack)	$16/mo
Phase 2: Live meetings	5–8	+$0 (100ms free tier)	$16/mo
Phase 3: Coding tests	9–12	+$0 (Judge on same VPS)	$16/mo
Phase 4: Mobile app	13–16	+$25 one-time (Google Play)	$16/mo + $25
Phase 5: SaaS white-label	17–20	+$2/mo (Cloudflare for SaaS)	$18/mo

13.4 First-Year Total Cost

Item	Cost
12 months × $16/mo (day-to-day base)	$192
Cloudflare for SaaS (8 months of Phase 5+)	$16
Google Play Store fee	$25
Apple App Store (if needed)	$99
Domain registration	$12
4 campaign months × $5 VPS premium	$20
AI analysis (12 months)	~$3
First-year total (without Apple)	**~$268**
First-year total (with Apple)	**~$367**

---

14. Scaling Playbook

14.1 Vertical Scaling (Hourly Billing)

Hetzner allows resizing with a 1–2 minute reboot. You pay only the hourly rate for the tier you're on.

Stage	VPS	Specs	Monthly (full)	Hourly	When
Day-to-day	CX32	4 vCPU, 8 GB, 80 GB	€13.99	€0.0224	Default
Campaign prep	CX42	8 vCPU, 16 GB, 160 GB	€25.49	€0.0408	Day before campaign
Heavy campaign	CCX23 (dedicated)	4 vCPU, 16 GB, 80 GB	€36.49	€0.0584	Only if shared CPU isn't enough
Post-campaign	CX32	4 vCPU, 8 GB, 80 GB	€13.99	€0.0224	Day after campaign ends

Campaign scaling procedure:

1. Day before campaign: `hcloud server change-type your-server cx42` (1–2 min downtime)
2. Run the 3–5 day campaign
3. Day after: `hcloud server change-type your-server cx32` (1–2 min downtime)
4. Total extra cost: ~€5 (~$5.50)

14.2 When to Move Beyond Single VPS

Signal	Action
Sustained CPU >70% on CX42	Split: API on VPS 1, Judge on VPS 2
PostgreSQL data >50 GB	Migrate to Neon Pro ($19/mo) or Supabase
>5 concurrent code submissions causing delays	Add second Judge worker VPS
>50 tenants (Phase 5 SaaS growth)	Move to Tier B architecture from v1.0 plan
Revenue >$2,000/month	Invest in managed services for reliability

14.3 Horizontal Scaling Path

```
Current (single VPS):
  [Caddy + API + Redis + Judge] + native PostgreSQL
  → €13.99/mo

Split 1 (API + Judge separation):
  VPS 1: [Caddy + API + Redis] + native PostgreSQL — CX22 (~€7)
  VPS 2: [Judge Worker] — CX22 (~€7)
  → ~€14/mo (same cost, better isolation)

Split 2 (managed DB):
  VPS 1: [Caddy + API + Redis] — CX22 (~€7)
  VPS 2: [Judge Worker] — CX22 (~€7)
  DB: Neon Pro ($19)
  → ~$33/mo
```

```
Full managed (v1.0 Tier B):
  API: Railway ($20)
  DB: Neon Pro ($19)
  Redis: Upstash ($10)
  Judge: Hetzner CX22 ($8)
  → ~$57/mo
```

---

15. ALTERNATE APPROACH: PaaS-Centric

15.1 Philosophy

Instead of managing a VPS, use Platform-as-a-Service providers for automatic deployments, scaling, and SSL. Trade higher cost for near-zero operational overhead. **You still need one VPS for the Judge service** (Docker execution requires a real server).

15.2 Architecture

```
┌──────────────────────────────────────────────────────────────────┐
│                      PaaS Architecture                           │
│                                                                  │
│  ┌──────────────┐  ┌──────────────┐                             │
│  │ Student Web  │  │ Admin Web    │  ← Vercel (Free)            │
│  │ (Vercel)     │  │ (Vercel)     │    or Cloudflare Workers    │
│  └──────┬───────┘  └──────┬───────┘                             │
│         └─────────┬───────┘                                      │
│                   ▼                                              │
│         ┌──────────────┐                                         │
│         │ API (Railway)  │  ← git push → auto deploy             │
│         │ Fastify + Node │                                       │
│         └──────┬───────┘                                         │
│                │                                                 │
│     ┌──────────┼──────────┐                                      │
│     ▼          ▼          ▼                                      │
│  ┌──────┐  ┌──────┐  ┌──────┐                                   │
│  │Neon  │  │Upstash│  │Hetzner │                                 │
│  │Postgr│  │Redis  │  │Judge VPS │  ← VPS still needed for Docker │
│  └──────┘  └──────┘  └──────┘                                   │
│                                                                  │
│  ┌──────────────────────────────────────┐                        │
│  │ Cloudflare R2 (file storage)          │                        │
│  └──────────────────────────────────────┘                        │
└──────────────────────────────────────────────────────────────────┘
```

15.3 Component Mapping & Costs

Component	Service	Monthly Cost	Notes

Component	Service	Monthly Cost	Notes
API	Railway (Pro, $5 credit)	$5–20	Usage-based after credit; auto-deploy on git push
Database	Neon Free → Launch ($19)	$0–19	Serverless Postgres; **cold starts** on free tier (5s delay after inactivity)
Redis	Upstash Free	$0–10	Serverless Redis, pay-per-request; free covers 10K commands/day
Judge Worker	Hetzner CX22	~€7 ($8)	Minimal VPS for Docker-based code execution
Student Web	Vercel Free or Cloudflare Workers	$0	100 GB bandwidth (Vercel), unlimited (Cloudflare)
Admin Web	Vercel Free or Cloudflare Workers	$0	Same
File Storage	Cloudflare R2	$0	Same as primary
AI Analysis	Gemini 2.0 Flash	~$0.15	Same
Email	Resend Free	$0	Same
Live Meetings	100ms Free	$0	Same
Monitoring	Sentry + Vercel Analytics	$0	Free tiers
TOTAL (day-to-day)		**$13–58/mo**	

> **Wide cost range** because Railway and Neon charges depend on actual usage. For 20 students: Railway ~$5–8/mo, Neon free tier likely sufficient. For campaigns: Railway auto-scales (more expensive) while Neon may need Launch plan ($19).

15.4 PaaS Pros and Cons

Factor	PaaS Advantage	PaaS Disadvantage
Deployment	`git push` → auto deploy	—
Scaling	Auto-scales (Railway)	More expensive at scale
SSL	Automatic	—
DB management	Managed (Neon)	Cold starts on free tier
DevOps effort	Near zero	—
Cost control	—	Harder to predict; usage-based pricing
Docker execution	—	Still need a VPS for Judge
Debugging	—	Limited shell access

Factor	PaaS Advantage	PaaS Disadvantage
Vendor lock-in	—	Moderate (Railway, Neon APIs)

16. Comparison: Primary vs Alternate

Dimension	Primary (VPS)	Alternate (PaaS)
Monthly cost (day-to-day)	**~$16**	$13–58
Monthly cost (campaign)	**~$21**	$25–70
First-year cost	**~$268**	$350–800
Deploy method	SSH + Docker Compose	**git push (auto)**
Cold starts	**None**	Yes (Neon, Railway sleep)
DB performance	**Native PostgreSQL, always on**	Serverless with cold starts
Scaling effort	Manual resize (1 min)	**Automatic**
Full server control	**Yes**	No
Judge service	Same VPS	Separate VPS still needed
Setup time	2–3 hours	**1 hour**
Maintenance	Monthly OS + Docker updates	**Near zero**
PostgreSQL in Docker?	**No (native)**	N/A (managed)
Campaign scale-down savings	**Yes ($5 premium for 5 days)**	Automatic but less predictable
Suitable for Phase 5 SaaS	Yes	Yes

Choose Primary if: You're comfortable with basic Linux/Docker, want predictable costs, and want the best performance for the money.

Choose Alternate if: You want zero DevOps, don't mind variable costs, and prefer git-push deploys over SSH scripts.

17. Pre-Deployment Checklist

#	Task	Approach	Priority	Effort
1	Provision Hetzner CX32, install PostgreSQL 16 natively	Primary	Critical	1 hour
2	Install Docker + Docker Compose on VPS	Primary	Critical	30 min
3	Create `docker-compose.prod.yml` (API, Redis, Caddy, Judge)	Both	Critical	2 hours
4	Create API Dockerfile (multi-stage build)	Both	Critical	2 hours
5	Migrate `storage.ts` to Cloudflare R2 (S3 client)	Both	Critical	4 hours

#	Task	Approach	Priority	Effort
6	Deploy student-web and admin-web to Cloudflare Workers	Both	Critical	1 hour
7	Configure Caddy reverse proxy with automatic SSL	Primary	Critical	1 hour
8	Set up PostgreSQL backup cron to R2	Primary	Critical	1 hour
9	Configure PostgreSQL `pg_hba.conf` for Docker bridge network	Primary	Critical	30 min
10	Integrate Resend for email invites	Both	Critical	2 hours
11	Integrate Gemini 2.0 Flash for AI analysis	Both	High	4 hours
12	Build Judge worker with BullMQ queue processing	Both	High	2–3 days
13	Pre-pull Docker language images on VPS	Both	High	30 min
14	Integrate 100ms/Dyte React SDK for live meetings	Both	High	1–2 days
15	Add health check endpoints (`/health`, `/ready`)	Both	High	30 min
16	Set up Sentry (API + frontends)	Both	High	1 hour
17	Set up BetterStack uptime monitoring	Both	High	30 min
18	Configure VPS firewall (allow 80, 443, 22 only)	Primary	High	30 min
19	Add Capacitor to student-web	Both	Medium	1–2 days
20	Publish to Google Play Store	Both	Medium	1 day
21	Set up GitHub Actions CI/CD pipeline	Both	Medium	2 hours
22	Build tenant branding admin UI (logo, colors, domain)	Both	Medium	2–3 days
23	Configure Cloudflare for SaaS (custom domains)	Both	Low (Phase 5)	2 hours
24	Integrate Razorpay/Stripe for tenant billing	Both	Low (Phase 5)	3–5 days

18. Revision History

Version	Date	Author	Changes
1.0	2026-02-24	Engineering Team	Initial draft — tiered SaaS deployment plan

Version	Date	Author	Changes
2.0	2026-03-04	Engineering Team	Complete rewrite: budget-optimized, phase-proof plan. PostgreSQL native (not Docker). Cloudflare Workers replaces Pages. Cloud provider comparison (Hetzner vs DO vs AWS vs GCP). Per-client branding details for Phase 5. Campaign scaling with hourly billing math. Primary (VPS) and alternate (PaaS) approaches with detailed pricing.